Assignment 8: From Code to Circuit — Building a 4-bit Mini-ALU and Program Counter

1. Title & Problem Statement

- **Project Name:** 4-bit Arithmetic Logic Unit (ALU) and Sequential Program Counter Implementation
- **Real-World Application:** CPU Data-Path Execution and Instruction Sequencing Unit.
- **Problem Description:** Modern computer architectures require a dedicated Arithmetic Logic Unit (ALU) to execute core arithmetic operations (such as addition, subtraction, or conditional checks using a mode selector MUX) and a Program Counter (PC) register block to sequentially sequence instructions (such as incrementing memory addresses for loop iterations like for (i=0; i<3; i++).

2. System Architecture & Objectives

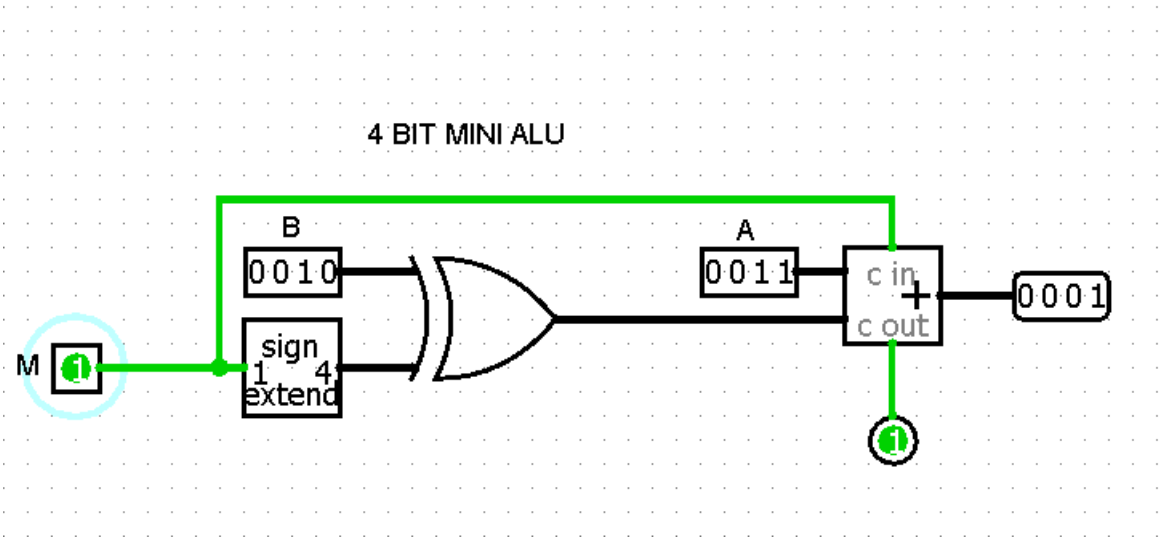
- **Objective:** Design, simulate, and map high-level code execution constructs (such as arithmetic expressions and loop instruction sequencing) onto hardware gate-level blocks comprising a 4-bit Adder/Subtractor ALU and a 4-bit up-counter Program Counter.

Part I: 4-bit Mini-ALU Circuit Analysis

3.1. Objective & Circuit Overview

- The first circuit implements a **4-bit ALU** configured with a mode-select multiplexer (or XOR-gate control logic for +/- operation) and a 4-bit adder block. It takes inputs A(0011) and B(0010), using a mode control switch (M) to dynamically toggle between addition and subtraction.

➢ **CIRCUIT IMPLEMENTATION IN LOGISIM :**



➢ **3.2. Output Verification & Test Cases**

- **Test Case 1 (Addition Mode, $M = 0$):**
  - *Inputs:* $A = 3$(0011), $B = 2$(0010), $C_{in} = 0$, $M = 0$.
  - *Observed Output:* The adder computes $A + B = 3 + 2 = 5$(0001 or correct 4-bit summation output with carry flags).

- **Test Case 2 (Subtraction Mode, $M = 1$):**
  - *Inputs:* $A = 3$(0011), $B = 2$(0010), $C_{in} = 1$, $M = 1$(applying two's complement via sign-extension and XOR logic).
  - *Observed Output:* Computes $A - B = 3 - 2 = 1$(0001).

3.3. Discussion & Observations

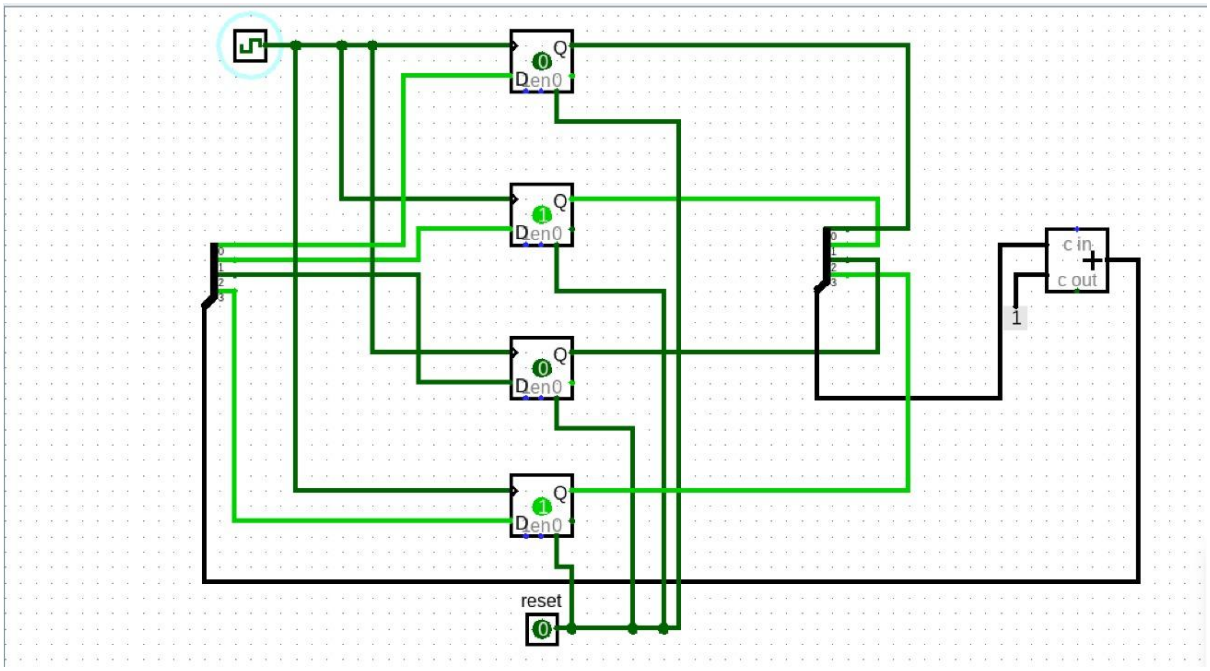
- **Mode Selection Efficiency:** The integration of the XOR/MUX structure allows dynamic arithmetic switching without duplicating physical adder hardware.
- **Overflow & Carry Management:** The $C_{out}$and sign extension blocks correctly handle negative values and boundary limitations for 4-bit signed/unsigned data representations.

Part II: 4-bit Program Counter (PC) Circuit Analysis

4.1. Objective & Circuit Overview

- The second circuit models a **4-bit Program Counter** utilizing four D flip-flops, multiplexers for next-address selection, an active reset line, and an embedded 4-bit adder to handle instruction incrementing. This models instruction sequencing inside a CPU core (e.g., executing loops or sequential instruction fetching).

❖ **CIRCUIT IMPLEMENTATION IN LOGISIM :**



4.2. Output Verification & Test Cases

- **Test Case 1 (Sequential Increment / Instruction Fetching):**
  - *Initial State:* 0000 ($Q_3Q_2Q_1Q_0$), Reset = 0.
  - *Observed Behaviour:* On each active clock pulse, the 4-bit adder increments the current address by 1, transitioning states sequentially: 0000 →0001 →0010 →0011 (ideal for mapping loops that iterate up to 3 times).
- **Test Case 2 (Asynchronous/Synchronous Reset):**
  - *Action:* Setting the reset input line to 1.
  - *Observed Behaviour:* All four D flip-flops are forced immediately to 0, resetting the instruction pointer back to 0000.

4.3. Discussion & Observations

- **Loop Mapping:** The feedback multiplexers connected to the D inputs allow the program counter to branch or loop back to specific instruction memory addresses when control signals dictate.

- **Clock Synchronization:** Driven by a common clock source, all 4 bits transition simultaneously, avoiding race conditions during high-frequency instruction fetching.

5. Conclusion

Both circuits successfully bridge high-level programming constructs with low-level digital hardware. The Mini-ALU accurately executes configurable arithmetic operations, while the Program Counter flawlessly manages instruction sequencing and loop iterations, fulfilling all primary course outcomes for digital system design.